



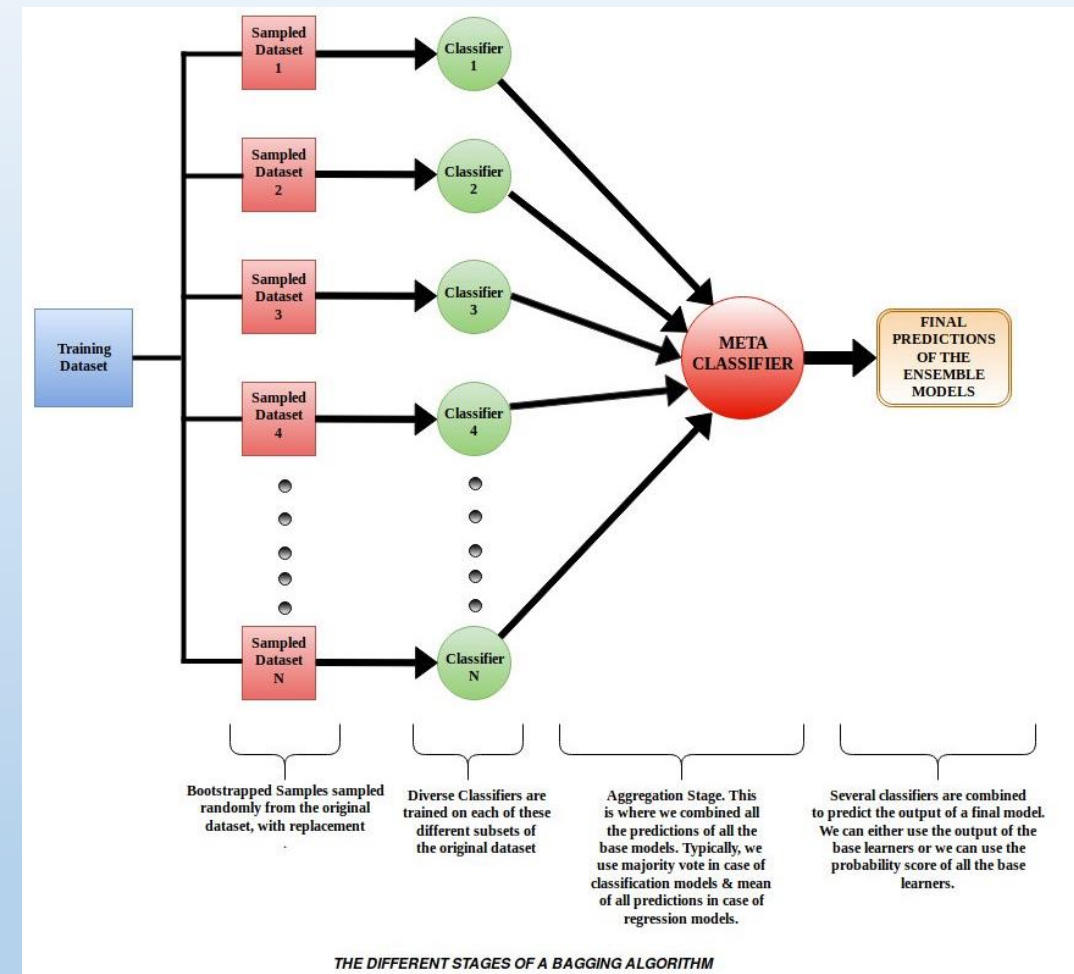
Ensemble Learning and Random Forest Algorithm



Data
Science

Ensemble is a **machine learning model that combines the predictions from two or more models**. The models that contribute to the ensemble, referred to as ensemble members.

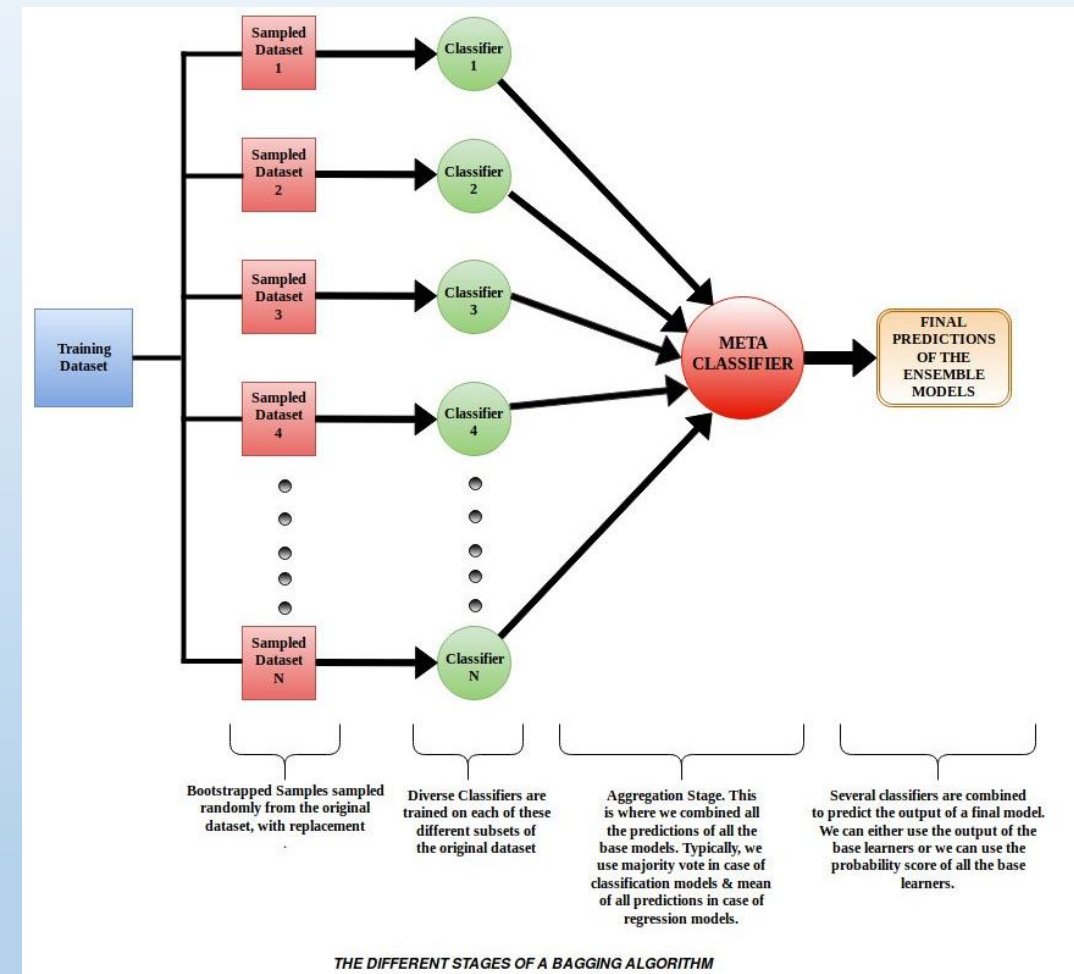
Ensemble learning is **the process by which multiple models, such as classifiers or experts, are strategically generated and combined to solve a particular computational intelligence problem**.



Ensemble Techniques:

1. Max Voting
2. Averaging
3. Bagging

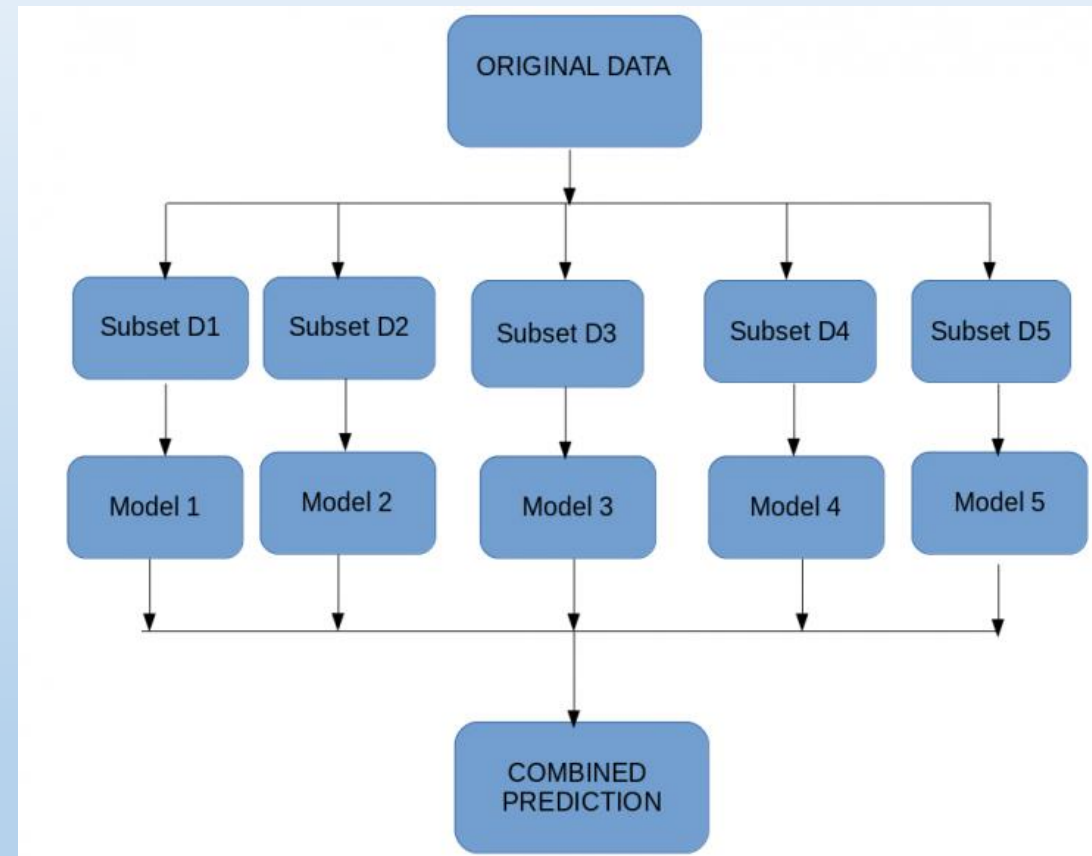
Bagging algorithms: Random forest



Ensemble Techniques:

1. Max Voting
2. Averaging
3. Bagging

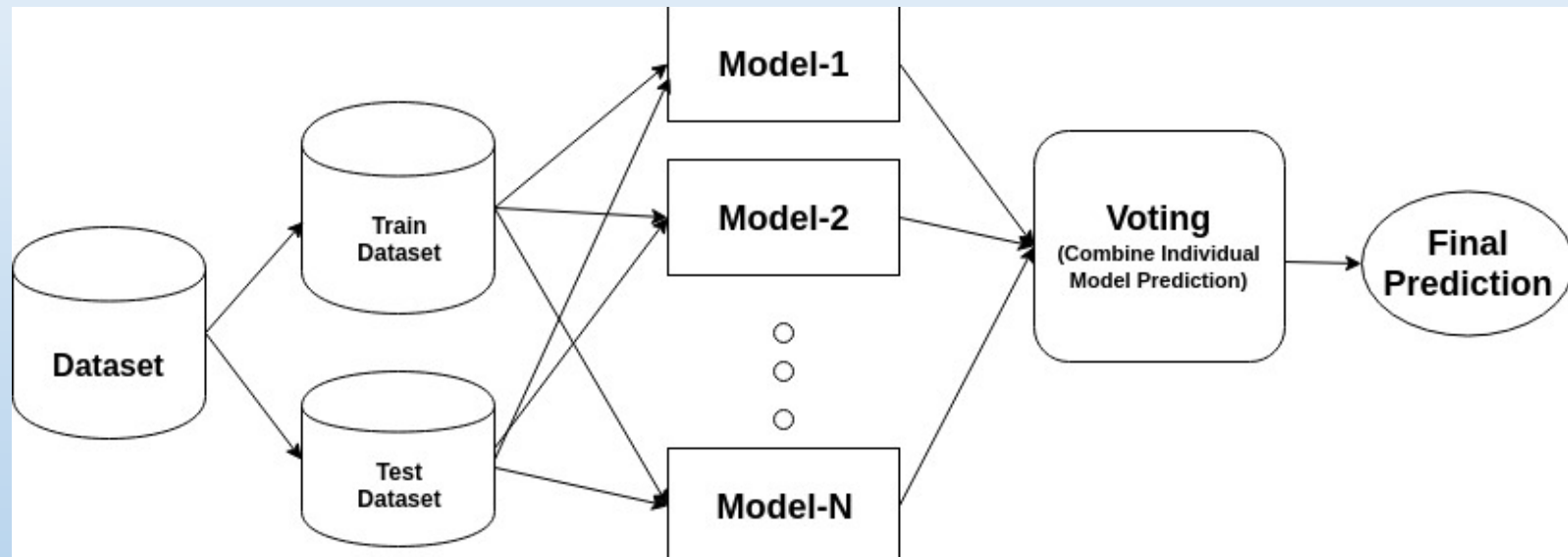
Bagging algorithms: Random forest



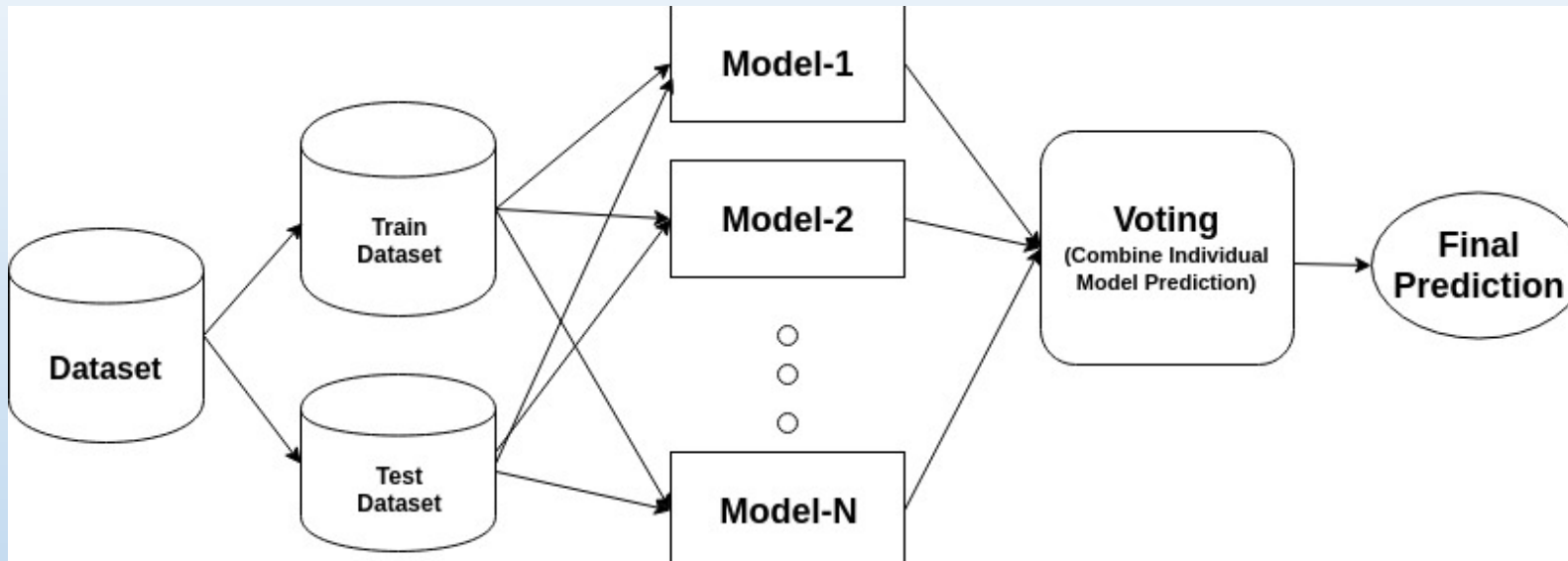
Ensemble Techniques

Ensemble Techniques:

1. Max Voting:
 - A. The max voting method is generally used for classification problems. In this technique, multiple models are used to make predictions for each data point.
 - B. The predictions which we get from the majority of the models are used as the final prediction.



Max Voting



```
from sklearn.ensemble import VotingClassifier
max_vote=VotingClassifier(estimators=[('random Forest',rm),('Logistic Regression',l1),('KNN:',kn)])
max_vote.fit(x_train,y_train)
y_pred_max=max_vote.predict(x_test)
print(y_pred_max)
ab=max_vote.score(x_test,y_test)*100
print("My Accuracy Score: ",ab)
```

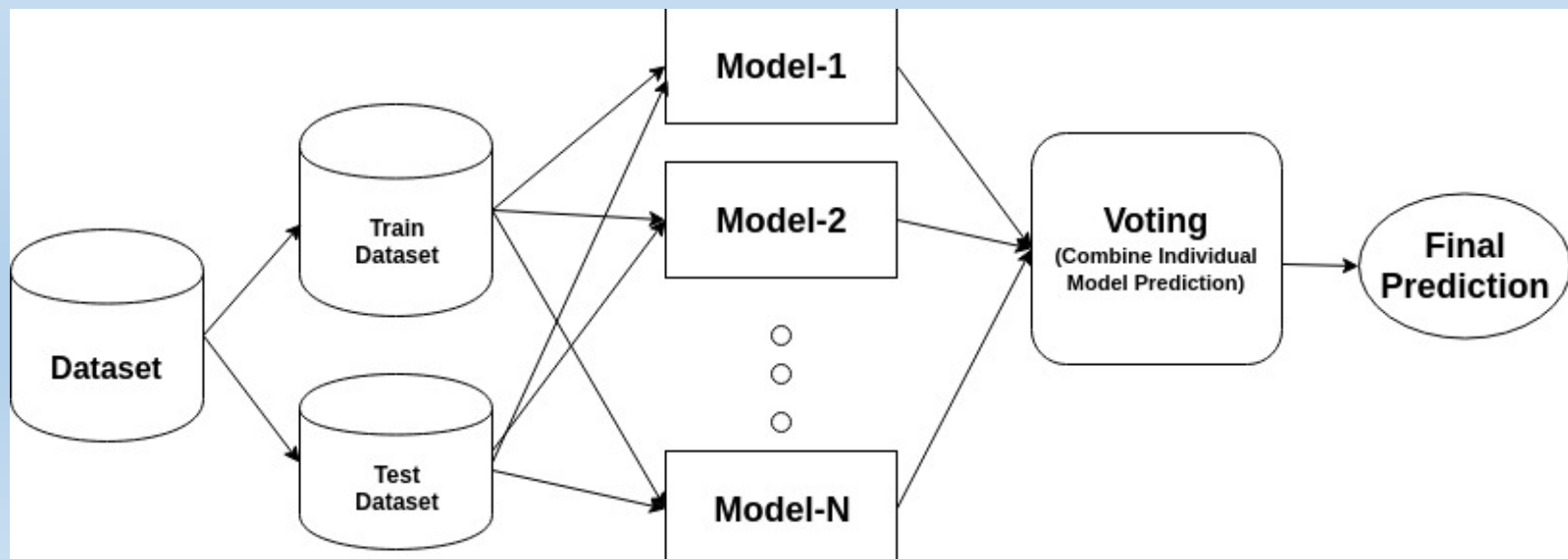
Max voting types

Hard voting .

Hard voting entails picking the prediction with the highest number of votes,

Soft voting

Soft voting entails combining the probabilities of each prediction in each model and picking the prediction with the highest total probability.

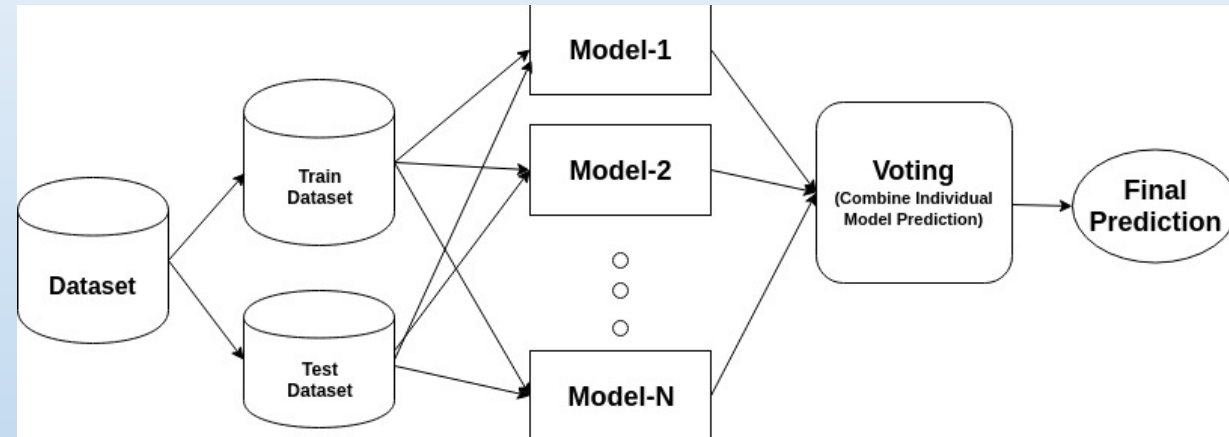


Weighted Average

Ensemble Techniques:

Weighted Average

- A. All models are assigned different weights defining the importance of each model for prediction. For instance, if two of your colleagues are critics, while others have no prior experience in this field, then the answers by these two friends are given more importance as compared to the other people..



```
# Averaging
final_vote=(y_pred1+y_pred2+y_pred3)/3
print(np.round(final_vote))
```

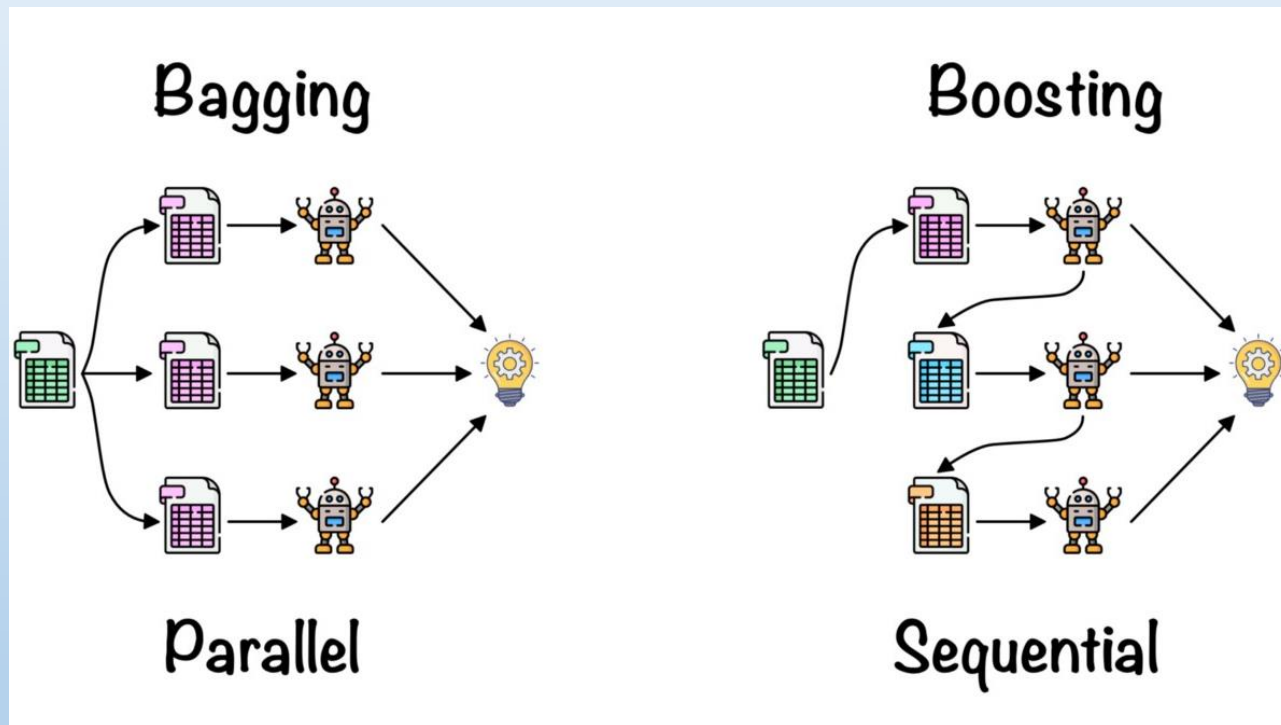
Bagging

Bagging :

Bagging is used when the goal is to reduce the variance of a decision tree classifier.

The objective is to create several subsets of data from training sample chosen randomly with replacement. Each collection of subset data is used to train their decision trees.

Average of all the predictions from different trees are used which is more robust than a single decision tree classifier.



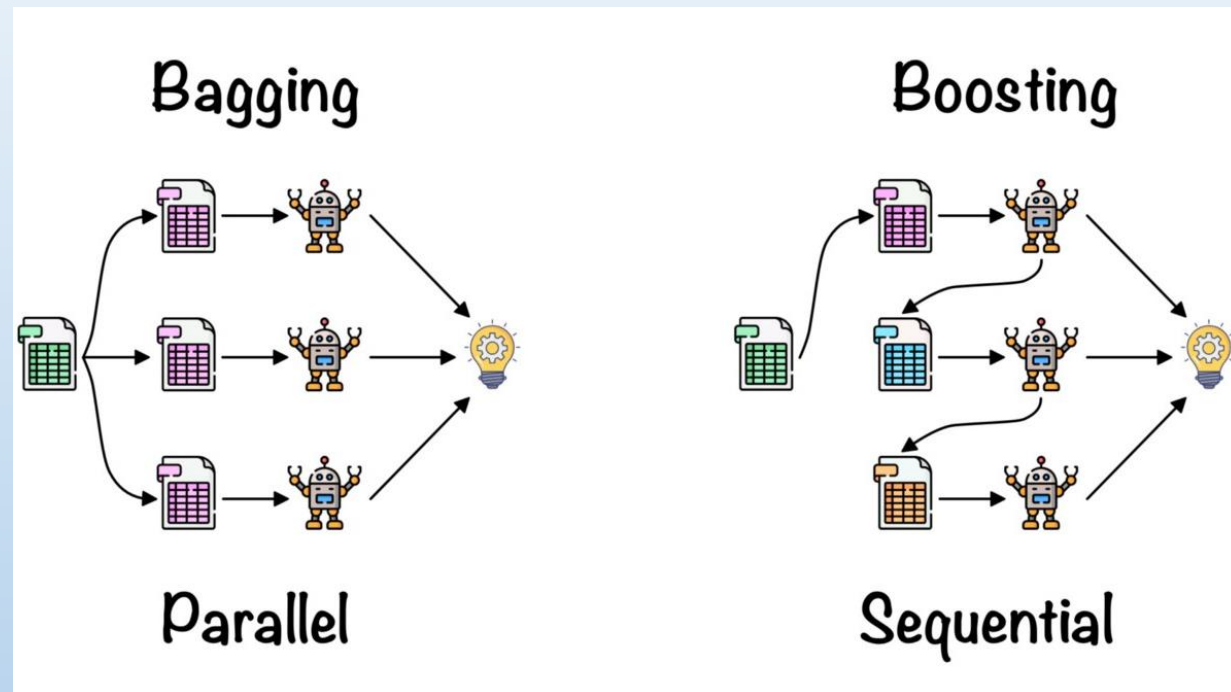
Bagging Advantages

Advantages:

- Reduces over-fitting of the model.
- Handles higher dimensionality data very well.
- Maintains accuracy for missing data.

Disadvantages:

- Since final prediction is based on the mean predictions from subset trees, it won't give precise values for the classification and regression model.



Bagging Python Syntax

Bagging Python Syntax :

```
from sklearn.ensemble import BaggingClassifier
bg=BaggingClassifier(base_estimator=l1,n_estimators=5,random_state=1)
bg.fit(x_train,y_train)
y_pred_bg=bg.predict(x_test)
print(y_pred_bg)
```

- `rfm = RandomForestClassifier(n_estimators=80, oob_score=True, n_jobs=-1, random_state=101, max_features = 0.50, min_samples_leaf = 5)`
- `fit(x_train, y_train)`
- `predicted = rfm.predict_proba(x_test)`

Boosting

Boosting :

Boosting is used to create a collection of predictors.

This technique, learners are learned sequentially with early learners fitting simple models to the data and then analyzing data for errors. Consecutive trees (random sample) are fit and at every step, the goal is to improve the accuracy from the prior tree.

Advantages:

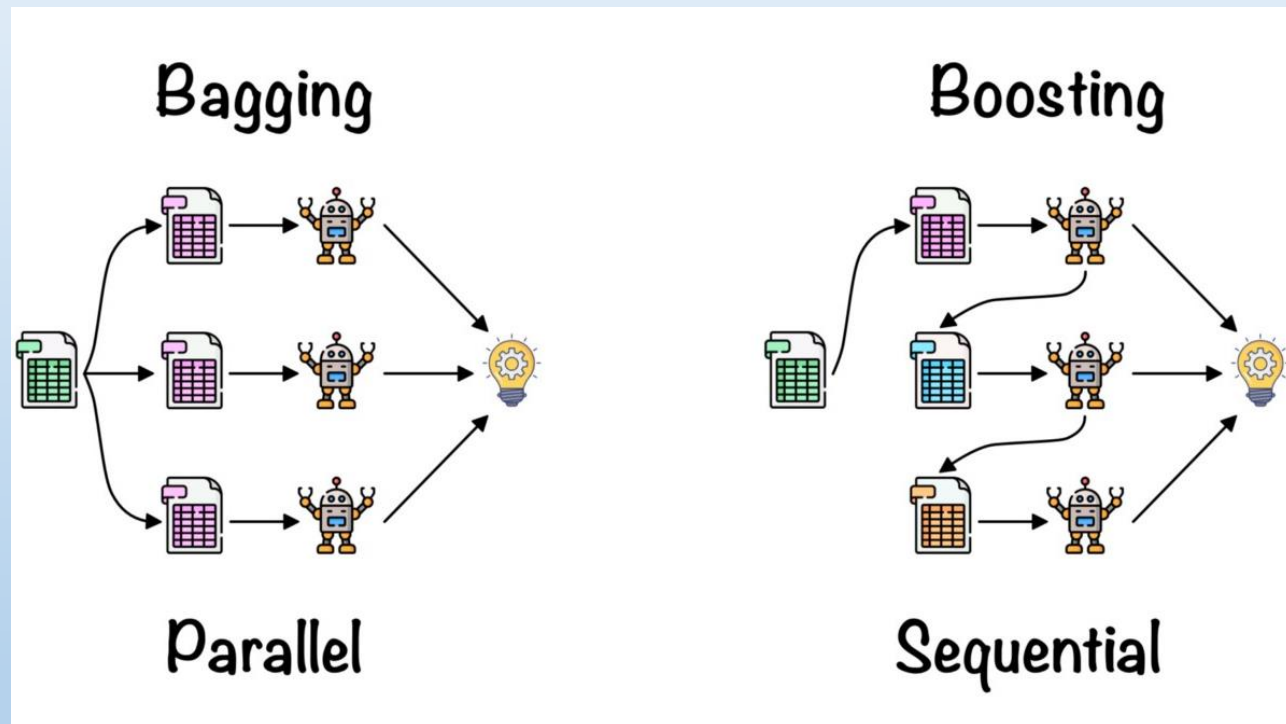
Supports different loss function (we have used 'binary: logistic' for this example).

Works well with interactions.

Disadvantages:

Prone to over-fitting.

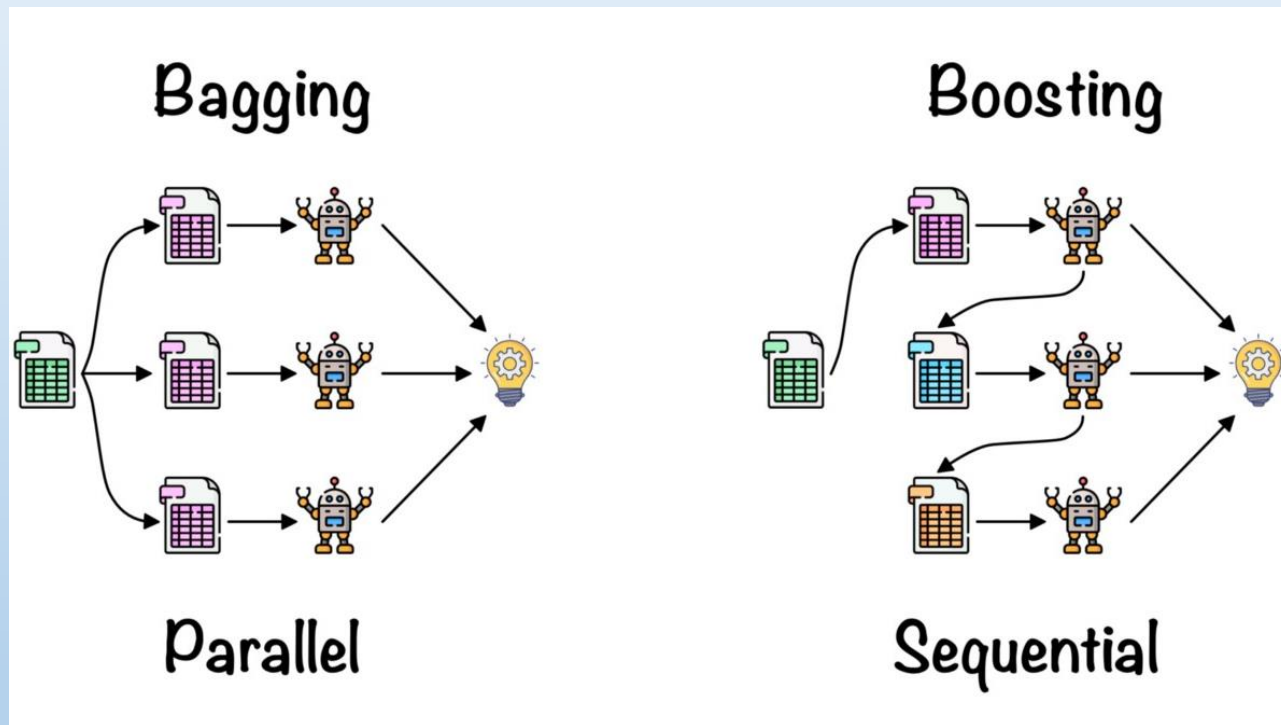
Requires careful tuning of different hyper-parameters.



Boosting Types

Boosting

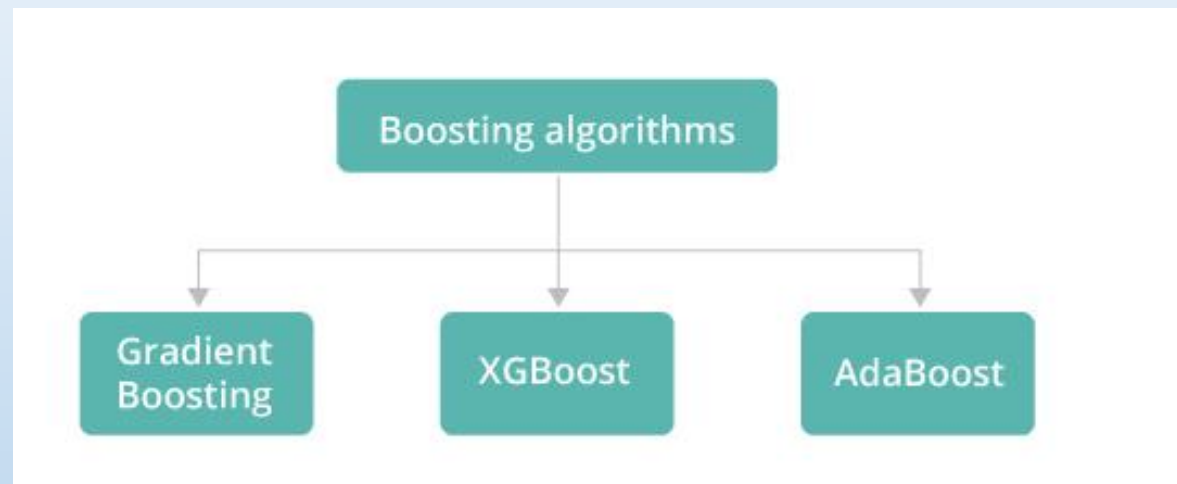
- AdaBoost
- GBM
- XGB



Ensemble Techniques

Boosting **Ada-Boost**

1. Adaptive boosting or AdaBoost is one of the simplest boosting algorithms.
2. Decision trees are used for modelling. Multiple sequential models are created, each correcting the errors from the last model.
3. AdaBoost assigns weights to the observations which are incorrectly predicted and the subsequent model works to predict these values correctly.



Ensemble Techniques

Boosting **Ada-Boost**

1. Adaptive boosting or AdaBoost is one of the simplest boosting algorithms.
2. Decision trees are used for modelling. Multiple sequential models are created, each correcting the errors from the last model.
3. AdaBoost assigns weights to the observations which are incorrectly predicted and the subsequent model works to predict these values correctly.

```
from sklearn.ensemble import AdaBoostClassifier
model = AdaBoostClassifier(random_state=1)
model.fit(x_train, y_train)
model.score(x_test, y_test)
0.81081081081081086
```

Sample code for regression problem:

```
from sklearn.ensemble import AdaBoostRegressor
model = AdaBoostRegressor()
model.fit(x_train, y_train)
model.score(x_test, y_test)
```

Ensemble Techniques

Boosting Python Syntax

- `from xgboost import XGBClassifier`
- `xgb = XGBClassifier(objective='binary:logistic', n_estimators=70, seed=101)`
- `fit(x_train, y_train)`
- `predicted = xgb.predict_proba(x_test)`